OOP: Object Oriented Programming

What is OOP?

Object-Oriented Programming (OOP) is a programming paradigm where we organize code around objects instead of just functions and logic.

An **object** represents a real-world entity with:

- **State (Data / Properties)** → What it *has*
- **Behavior (Methods / Functions)** → What it *does*

👉 Instead of writing scattered logic, OOP models software like the real world.

Think about a **Car**.

Real World	**In Code**
Car	Object
Color, Speed, Brand	Properties
Start(), Brake()	Methods

We don't program "steps."
We create a **Car object** and tell it what to do.

Why OOP Was Needed

Before OOP (Procedural Programming):

❌ Code became:

- Hard to scale
- Difficult to maintain
- Repetitive
- Bug-prone

Example of procedural thinking:

```
let car1Name = "BMW";
let car1Speed = 120;

let car2Name = "Audi";
let car2Speed = 150;
```

This doesn't scale.

OOP solves this by introducing **Blueprints**.

Class = Blueprint of Objects

A **Class** is a template used to create objects.

```
class Car {
constructor(name, speed) {
this.name = name;
this.speed = speed;
}

drive() {
console.log(this.name + " is driving at " + this.speed);
}
}
```

Now create objects:

```
const car1 = new Car("BMW", 120);
const car2 = new Car("Audi", 150);
```

We reused ONE blueprint to create many objects.

The 4 Pillars of OOP

1️⃣ Encapsulation (Data Protection)

Encapsulation means:

Bundle data + methods together and restrict direct access.

2️⃣ Abstraction (Hide Complexity)

Abstraction means:

Show only what is necessary, hide how it works internally.

3️⃣ Inheritance (Reusability)

Inheritance allows one class to reuse another class's behavior.

4️⃣ Polymorphism (Many Forms)

Polymorphism means:

Same method behaves differently for different objects.

Important Clarification (Especially for JavaScript Developers)

In many languages OOP is class-based.

But in JavaScript:

👉 OOP is built on **Prototypes** (not true classes).
👉 class is just syntactic sugar over prototype inheritance.

So learning OOP + prototypes together gives deeper mastery.

Prototypes

Prototypes are the core inheritance mechanism in JavaScript.

Unlike classical OOP (Java/C++), JavaScript does NOT copy behavior — it links objects together; This is called **Prototypal Inheritance**.

1. The Problem Prototypes Solve

Imagine creating 1000 users:

```
function createUser(name) {
 return {
  name,
  greet() {
   console.log("Hello " + this.name);
  }
 };
}
```

Every time you call createUser, a **new copy of greet()** is created.
That means:
❌ 1000 users = 1000 greet functions (memory waste)

2. Prototype = Shared Memory for Objects

Instead of copying methods,
JavaScript stores them in a **prototype object** that all instances reference.
Think:
Objects don't own methods → they borrow them.

3.Every JavaScript Object Has a Hidden Link

Every object has an internal pointer:
➔ [[Prototype]]
➔ Object.getPrototypeOf(obj)
➔ obj.__proto__

this pointer forms the Prototype Chain.

How Functions Create Prototypes

Every function automatically gets a .prototype property.

```
function User(name) {
  this.name = name;
}

console.log(User.prototype);
```

Output:

```
{
  constructor: User
}
```

This object is where we store shared behavior.

. Adding Methods to Prototype (The Correct Way)

```
User.prototype.greet = function () {
  console.log("Hello " + this.name);
};
```

Now create objects:

```
const u1 = new User("Prajwal");
const u2 = new User("Rahul");
```

Neither object contains greet.

Instead:

```
u1 → [[Prototype]] → User.prototype → greet()
u2 → [[Prototype]] → User.prototype → greet()
```

✓ ONE function shared by ALL instances.

What new Actually Does (Important Interview Question)

new User("Prajwal") is equivalent to:

```
const obj = {};
obj.__proto__ = User.prototype;   // Link created
User.call(obj, "Prajwal");        // Constructor runs
return obj;
```

So new:

1️⃣ Creates empty object
2️⃣ Links it to prototype
3️⃣ Runs constructor with this
4️⃣ Returns object

Property Lookup — How JavaScript Searches

```
u1.greet();
```

JS looks in this order:

Step 1 → Does u1 have greet? ❌
Step 2 → Look inside User.prototype ✅
Step 3 → If not found → Object.prototype
Step 4 → If not found → null (stop)

This is the **Prototype Chain**:

u1
↓
User.prototype
↓
parent.prototype
↓
Object.prototype

↓
null

This is Why All Objects Can Use .toString()

Because toString() lives here:

Object.prototype

So every object inherits it automatically.

9. Overriding Prototype Methods

If instance defines same method:

```
u1.greet = function () {
  console.log("Custom greet");
};
```

Now lookup finds it at instance level first.

u1 (own method) → STOP

Prototype method is ignored.

This is called **Shadowing**.

10. Changing Prototype After Creation (Danger Zone)

```
User.prototype.sayBye = function () {
  console.log("Bye");
};
```

Even existing objects get access:

```
u1.sayBye(); // works
```

Because they are linked, not copied.

Difference Between prototype vs __proto__

This confuses many developers:

Thing	Exists On	Purpose
function.prototype	Functions	Where shared methods live
object.__proto__	All objects	Points to its prototype

They are connected like this:

```
instance.__proto__ === Constructor.prototype  // true
```

ES6 class is Just Prototype Sugar

This:

```
class User {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log("Hello " + this.name);
  }
}
```

Is internally converted to:

```
function User(name) {
  this.name = name;
}
```

```
User.prototype.greet = function () {
  console.log("Hello " + this.name);
};
```

So:

JavaScript never had real classes. Only prototypes.

14.Real Inheritance Using Prototypes

```
function Admin(name) {
  User.call(this, name);
}

Admin.prototype = Object.create(User.prototype);
Admin.prototype.constructor = Admin;
```

Now chain becomes:

admin
↓
Admin.prototype
↓
User.prototype
↓
Object.prototype

Admin inherits User behavior **without copying**.

14. Why Prototypes Matter in Real Development

Understanding prototypes helps you:

- ✅ Debug weird this bugs
- ✅ Understand performance of classes
- ✅ Avoid memory leaks
- ✅ Write better abstractions

Mental Model (The One That Makes It Click)

Think of objects like employees in a company.

They don’t carry the rulebook.

They know **where the rulebook is kept**.

Object → asks Prototype → asks Parent Prototype → asks Object.prototype

They follow references, not copies.

A prototype is an object that other objects link to for shared behavior.

Questions

Q. Difference Between OOP and Procedural Programming?

OOP	Procedural
Object-based	**Function-based**
Reusable	**Repetitive**
Encapsulated	**Data exposed**
Scalable	**Hard to scale**

Q.What is a Prototype in JavaScript?

Every JavaScript object has a hidden internal property:

[[Prototype]]

It links to another object from which it can inherit properties and methods.

👉 **Instead of copying methods, JavaScript shares them through this link.**

Q. What is Prototypal Inheritance?

Objects inherit directly from other objects.

```
const parent = { greet() { console.log("Hello"); } };

const child = Object.create(parent);
child.greet();
```

child doesn’t have greet, but it finds it via prototype chain.

Q. Difference Between Classical Inheritance and Prototypal Inheritance?

Classical (Java)	Prototype (JavaScript)
Class → Object	Object → Object
Copies behavior	Links behavior
Rigid	Dynamic

Q. What is Object.create()?

Creates a new object with a specified prototype.

```
const admin = Object.create(user);
```

Used for manual inheritance without constructors.